

MODULE 2 · CORE WORKLOADS

# Deployments & ReplicaSets

A controller that keeps the right number of Pods running, always

---

# Bare Pods don't self-heal — Deployments do

A bare Pod has nobody watching it: delete it, or lose its node, and it's simply gone. That's fine for a quick test, unacceptable for anything real.

A **Deployment** closes that gap. You declare how many Pods you want and what they should look like; a controller continuously makes reality match that declaration — creating, replacing, and scaling Pods for you.

## BY THE END YOU CAN

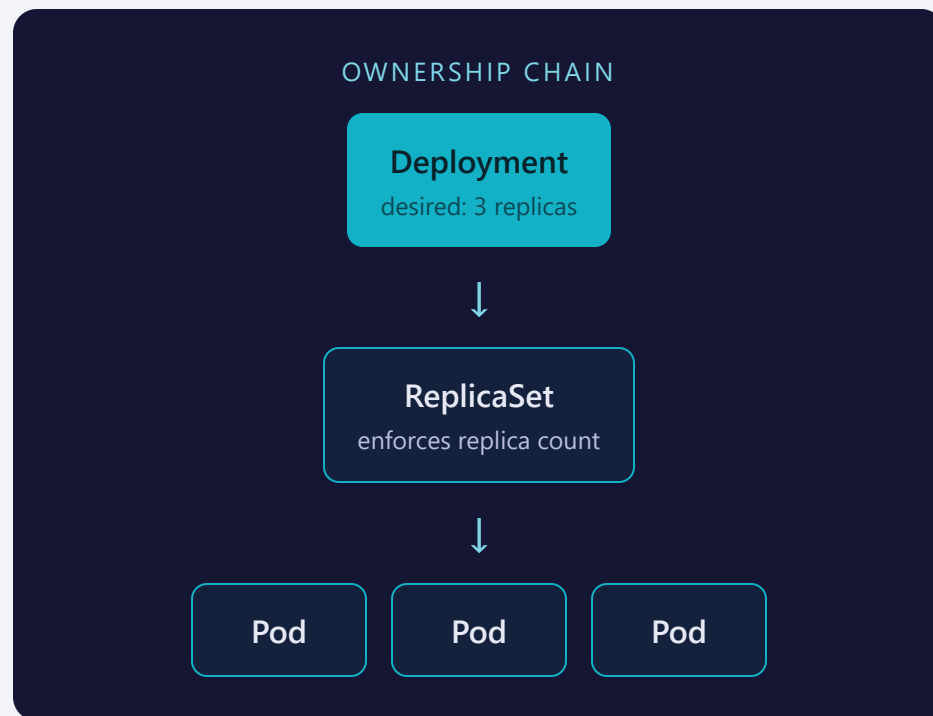
- Explain the Deployment → ReplicaSet → Pod chain
- Describe desired-vs-actual reconciliation
- Scale a Deployment up and down
- Say why lost Pods get replaced automatically

## CONCEPT

# Deployment owns ReplicaSet owns Pods

- **Deployment** — your declaration of intent: image, replica count, update strategy
- **ReplicaSet** — created by the Deployment; its one job is keeping exactly N matching Pods alive
- **Pods** — the actual running instances, created and deleted by the ReplicaSet

You almost never touch a ReplicaSet directly — you edit the Deployment and the chain below it follows.



## RECONCILIATION

# Desired state vs actual state, on a loop

RUNS CONTINUOUSLY, FOR EVERY REPLICASET

Watch actual Pods



Compare to spec.replicas



Create or delete until equal

### MENTAL MODEL

`spec.replicas` is a declaration, not a one-time command. Anything that changes the actual count — a scale, a deleted Pod, a crashed node — is noticed and corrected on the next loop.

# A Pod template, plus a replica count

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  namespace: training
  labels:
    app: web
spec:
  replicas: 3           # desired count
  selector:
    matchLabels:       # must match template labels
      app: web
  template:            # the Pod spec, stamped out N times
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

- **replicas** — the desired count the controller enforces
- **selector** — how the ReplicaSet finds "its" Pods
- **template** — a full Pod spec, stamped out N times

`selector.matchLabels` must match `template.metadata.labels`, or the object is rejected.

# Imperative to explore, declarative to keep

## IMPERATIVE — FAST, ONE-OFF

```
kubectl create deployment web --image=nginx --replicas=3
```

Builds the Deployment for you, submits it now

## DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f deployment.yaml
```

You own the file; it's reviewable and re-appliable

### THE BRIDGE TRICK

Generate the YAML without creating anything, then edit & keep it:

```
$ kubectl create deployment test-deploy --image=nginx --replicas=2 \
  --dry-run=client -o yaml
```

SEE IT

## The chain, visible in three commands

```
$ kubectl get deployment web-deployment -n training
NAME                READY   UP-TO-DATE   AVAILABLE
web-deployment      3/3     3             3

$ kubectl get rs -n training
NAME                DESIRED   CURRENT   READY
web-deployment-7d9f8c6b5    3         3         3

$ kubectl get pods -n training -l app=web
web-deployment-7d9f8c6b5-4kxpz    Running
web-deployment-7d9f8c6b5-h2j7q    Running
web-deployment-7d9f8c6b5-w9t1m    Running
```

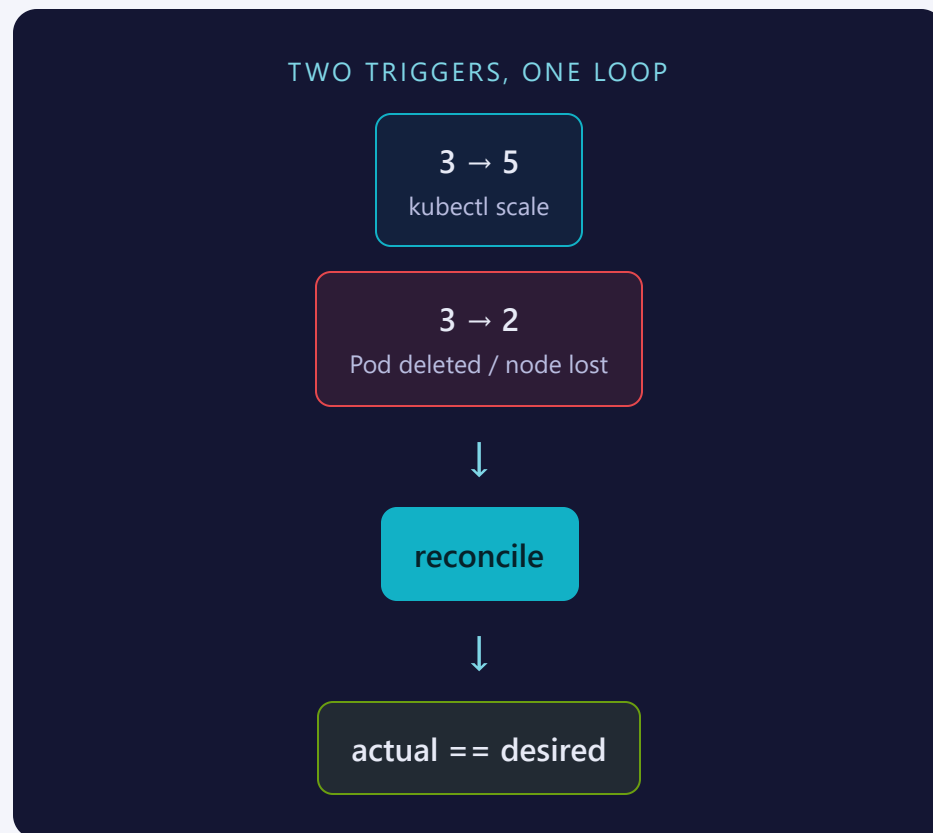
Every Pod name embeds its owning ReplicaSet's hash, then a unique suffix — the naming scheme *is* the ownership chain.

## Same loop, two different triggers

`kubectl scale` just edits `spec.replicas` on the live object — the same reconciliation loop then creates or deletes Pods to match.

Kill a Pod by hand, or lose it to a node failure, and nothing about the loop changes: actual count drops below desired, and a replacement appears within seconds — no human involved.

Scaling and self-healing aren't two features. They're the same control loop reacting to two different causes.





## Where people trip

- **Editing a Pod directly.** The ReplicaSet doesn't know about your edit — it just recreates the Pod from the template.
- **Selector/label mismatch.** `selector.matchLabels` must match the template's labels or the Deployment is rejected.
- **`kubectl scale` is imperative.** It changes the live object, not your YAML file — the next `apply` silently reverts it.
- **Reading only get deployments.** Check `get rs` and `get pods` too — that's where the real ownership shows up.

# Commands to keep at hand

```
$ kubectl get deployments -n NS           # ready / up-to-date / available
$ kubectl describe deployment NAME -n NS  # selector, template, events
$ kubectl get rs -n NS                   # the ReplicaSet(s) it owns
$ kubectl get pods -n NS -l app=LABEL     # the Pods, by label
$ kubectl scale deployment NAME --replicas=N -n NS # change desired count
$ kubectl delete deployment NAME -n NS    # deletes RS + Pods too
```

**Tip:** deleting a Deployment cascades — its ReplicaSet and Pods go with it.

## RECAP

# Deployments in one breath

- Deployment → ReplicaSet → Pods — an ownership chain, not one flat object
- A control loop continuously reconciles desired vs actual state
- `kubectl scale` changes desired count; the loop does the rest
- Lost or deleted Pods get replaced automatically — no bare-Pod gap

### HANDS-ON

Now run Lab 2.3  
[labs/2.3/](#)

Next: [2.4 — Rollouts & Rollbacks](#)